Book(bookid, bookname, publisher, price)

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구 아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	배구 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000

Orders(orderid, custid, bookid, saleprice, orderdate)

orderid	custid	bookid	saleprice	orderdate
1	1	1	6000	2024-07-01
2	1	3	21000	2024-07-03
3	2	5	8000	2024-07-03
4	3	6	6000	2024-07-04
5	4	7	20000	2024-07-05
6	1	2	12000	2024-07-07
7	4	8	13000	2024-07-07
8	3	10	12000	2024-07-08
9	2	10	7000	2024-07-09
10	3	8	13000	2024-07-10

Customer(custid, name, address, phone)

custid	name	address	phone
1	박지성	영국 맨체스터	000-5000-0001
2	김연아	대한민국 서울	000-6000-0001
3	김연경	대한민국 경기도	000-7000-0001
4	추신수	미국 클리블랜드	000-8000-0001
5	박세리	대한민국 대전	NULL

표 3-5 SELECT 문 문법({ }는 필수, [ ]는 선택, |는 선택 가능한 문법들 중 하나 사용 가능)

(a) 기본형	SELECT [ALL | DISTINCT] 속성이름(들) FROM 테이블이름(들) [WHERE 검색조건(들)] [GROUP BY 속성이름] [HAVING 검색조건(들)] [ORDER BY 속성이름 [ASC | DES]]
(b) 상세형	SELECT [ALL | DISTINCT] [테이블이름.]{ * | 속성이름 [[AS] 속성이름별칭]} [FROM {테이블이름 [AS 테이블이름별칭]} [INNER JOIN | LEFT [OUTER] JOIN | RIGHT [OUTER] JOIN {테이블이름 [ON 검색조건]}] [WHERE 검색조건(들)] [GROUP BY {속성이름, [..., n]}] [HAVING 검색조건(들)] [질의 UNION 질의 | 질의 UNION ALL 질의] [ORDER BY {속성이름 [ASC | DESC], [..., n]}]

표 3-6 WHERE 절에 조건으로 사용할 수 있는 술어

술어	연산자	사용 예
비교	=, <>, <, <=, >, >=	price < 20000
범위	BETWEEN	price BETWEEN 10000 AND 20000
집합	IN, NOT IN	price IN (10000, 20000, 30000)
패턴	LIKE	bookname LIKE '축구의 역사'
NULL	IS NULL, IS NOT NULL	price IS NULL
복합조건	AND, OR, NOT	(price < 20000) AND (bookname LIKE '축구의 역사')

표 3-7 와일드 문자의 종류

와일드 문자	의미	사용 예
%	0개 이상의 문자열과 일치	'%축구%': 축구를 포함하는 문자열
[ ]	1개의 문자와 일치	'[0–5]%': 0–5 사이 숫자로 시작하는 문자열
[^]	1개의 문자와 불일치	'[^0–5]%': 0–5 사이 숫자로 시작하지 않는 문자열
_	특정 위치에 있는 1개의 문자와 일치	'_구%': 두 번째 위치에 '구'가 들어가는 문자열

표 3-10 조인 문법

명령	문법	설명
일반적인 조인	SELECT 〈속성들〉 FROM 테이블 1, 테이블 2 WHERE 〈조인조건〉 AND 〈검색조건〉	SQL 문에서는 주로 동등조인을 사용한다. 두 가지 문법 중 하나를 사용할 수 있다.
	SELECT 〈속성들〉 FROM 테이블 1 INNER JOIN 테이블 2 ON 〈조인조건〉 WHERE 〈검색조건〉	
외부조인	SELECT 〈속성들〉 FROM 테이블 1 {LEFT | RIGHT | FULL [OUTER]} JOIN 테이블 2 ON 〈조인조건〉 WHERE 〈검색조건〉	외부조인이면 FROM 절에 조인 종류를 적고 ON을 이용하여 조인조건을 명시한다. FULL OUTER JOIN은 SQL 표준 문법에는 있지만 MySQL은 지원하지 않는다.

표 3-8 집계 함수의 종류

집계 함수	문법	사용 예
SUM	SUM([ALL | DISTINCT] 속성이름)	SUM(price)
AVG	AVG([ALL | DISTINCT] 속성이름)	AVG(price)
COUNT	COUNT({[[ALL | DISTINCT] 속성이름] | * })	COUNT(*)
MAX	MAX([ALL | DISTINCT] 속성이름)	MAX(price)
MIN	MIN([ALL | DISTINCT] 속성이름)	MIN(price)

표 3-11 데이터타입 종류

데이터 타입	설명	ANSI SQL 표준 타입
INTEGER INT	4바이트 정수형을 저장한다.	INTEGER, INT SMALLINT
NUMERIC(m,d) DECIMAL(m,d)	전체 자릿수 m, 소수점 이하 자릿수 d를 가진 숫자형을 저장한다.	DECIMAL(p, s) NUMERIC[(p,s)]
CHAR(n)	문자형 고정길이, 문자를 저장하고 남은 공간은 공백으로 채운다.	CHARACTER(n) CHAR(n)
VARCHAR(n)	문자형 가변길이를 저장한다.	CHARACTER VARYING(n) CHAR VARYING(n)
DATE	날짜형, 연도, 월, 일, 시간을 저장한다.	

```
CREATE TABLE 테이블이름
    ( {속성이름 데이터타입
        [NULL | NOT NULL | UNIQUE | DEFAULT 기본값 | CHECK 체크조건]
        }
        [PRIMARY KEY 속성이름(들)]
        [FOREIGN KEY 속성이름 REFERENCES 테이블이름(속성이름)]
            [ON DELETE {CASCADE | SET NULL}]
    )
```

```
DROP TABLE NewBook;
```

```
CREATE TABLE  NewOrders (
  orderid    INTEGER,
  custid     INTEGER     NOT NULL,
  bookid     INTEGER     NOT NULL,
  saleprice  INTEGER,
  orderdate  DATE,
  PRIMARY KEY(orderid),
  FOREIGN KEY(custid) REFERENCES NewCustomer(custid) ON DELETE CASCADE);
```

```
CREATE TABLE     NewBook (
   bookname      VARCHAR(20)      NOT NULL,
   publisher     VARCHAR(20)      UNIQUE,
   price         INTEGER  DEFAULT 10000 CHECK(price >= 1000),
   PRIMARY KEY   (bookname, publisher));
```

```
ALTER TABLE 테이블이름
   [ADD 속성이름 데이터타입]
   [DROP COLUMN 속성이름]
   [ALTER COLUMN 속성이름 데이터타입]
   [ALTER COLUMN 속성이름 [NULL | NOT NULL]]
   [ADD PRIMARY KEY(속성이름)]
   [[ADD | DROP] 제약이름]
```

```
ALTER TABLE NewBook ADD isbn VARCHAR(13);
```

```
ALTER TABLE NewBook MODIFY isbn INTEGER;
```

```
ALTER TABLE NewBook MODIFY bookname VARCHAR(20) NOT NULL;
```

```
ALTER TABLE NewBook DROP COLUMN isbn;
```

```
INSERT INTO 테이블이름[(속성리스트)]
       VALUES (값리스트);
```

```
INSERT INTO Book(bookid, bookname, publisher, price)
       VALUES (11, '스포츠 의학', '한솔의학서적', 90000);
```

```
UPDATE   테이블이름
SET      속성이름 1=값 1[, 속성이름 2=값 2, ...]
[WHERE   <검색조건>];
```

```
SET SQL_SAFE_UPDATES=0;   /* Safe Updates 옵션 미 해제 시 실행 */
UPDATE   Customer
SET      address ='대한민국 부산'
WHERE    custid=5;
```

```
UPDATE   Book
SET      publisher = (SELECT publisher
                      FROM   imported_book
                      WHERE  bookid = 21)
WHERE    bookid = 14;
```

```
DELETE FROM 테이블이름
[WHERE 검색조건];
```

```
DELETE  FROM    Book
WHERE   bookid = 11;
```

표 4-2 숫자 함수의 종류

함수	설명
ABS(숫자)	숫자의 절댓값을 계산함 예 ABS(-4.5) → 4.5
CEIL(숫자)	숫자보다 크거나 같은 최소의 정수 예 CEIL(4.1) → 5
FLOOR(숫자)	숫자보다 작거나 같은 최소의 정수 예 FLOOR(4.1) → 4
ROUND(숫자, m)	숫자의 반올림, m은 반올림 기준 자릿수 예 ROUND(5.36, 1) → 5.40
LOG(n, 숫자)	숫자의 자연로그 값을 반환함 예 LOG(10) → 2.30259
POWER(숫자, n)	숫자의 n 제곱 값을 계산함 예 POWER(2, 3) → 8
SQRT(숫자)	숫자의 제곱근 값을 계산함(숫자는 양수) 예 SQRT(9.0) → 3.0
SIGN(숫자)	숫자가 음수이면 -1, 0이면 0, 양수이면 1 예 SIGN(3.45) → 1

표 4-3 문자 함수의 종류(s: 문자열, c: 문자, n과 k: 정수)

문자값 반환 함수	설명
CONCAT(s1,s2)	두 문자열을 연결함 예 CONCAT('마당', '서점') → '마당서점'
LOWER(s)	대상 문자열을 모두 소문자로 변환함 예 LOWER('MR. SCOTT') → 'mr. scott'
LPAD(s,n,c)	대상 문자열의 왼쪽부터 지정한 자릿수까지 지정한 문자로 채움 예 LPAD('Page 1', 10, '*') → '****Page 1'
REPLACE(s1,s2,s3)	대상 문자열의 지정한 문자를 원하는 문자로 변경함 예 REPLACE('JACK & JUE', 'J', 'BL') → 'BLACK & BLUE'
RPAD(s,n,c)	대상 문자열의 오른쪽부터 지정한 자릿수까지 지정한 문자로 채움 예 RPAD('AbC', 5, '*') → 'AbC**'
SUBSTR(s,n,k)	대상 문자열의 지정된 자리에서부터 지정된 길이만큼 잘라서 반환함 예 SUBSTR('ABCDEFG', 3, 4) → 'CDEF'
TRIM(c FROM s)	대상 문자열의 양쪽에서 지정된 문자를 삭제함(문자열만 넣으면 기본값으로 공백 제거) 예 TRIM('=' FROM '==BROWNING==') → 'BROWNING'
UPPER(s)	대상 문자열을 모두 대문자로 변환함 예 UPPER('mr. scott') → 'MR. SCOTT'
숫자값 반환 함수	**설명**
ASCII(c)	대상 알파벳 문자의 아스키코드 값을 반환함 예 ASCII('D') → 68
LENGTH(s)	대상 문자열의 byte를 반환함(알파벳은 1byte, 한글은 3byte (UTF-8)) 예 LENGTH('CANDIDE') → 7
CHAR_LENGTH(s)	문자열의 문자 수를 반환함 예 CHAR_LENGTH('데이터') → 3

표 4-4 날짜 · 시간 함수의 종류

함수	반환형	설명
STR_TO_DATE(string, format)	DATE	문자열(STRING) 데이터를 날짜형(DATE)으로 반환함 예 STR_TO_DATE('2024-12-07', '%Y-%m-%d') → 2024-12-07
DATE_FORMAT(date, format)	STRING	날짜형(DATE) 데이터를 문자열(VARCHAR)로 반환함 예 DATE_FORMAT('2024-12-07', '%Y-%m-%d') → '2024-12-07'
ADDDATE(date, interval)	DATE	DATE형의 날짜에서 INTERVAL 지정한 시간만큼 더함 예 ADDDATE('2024-12-07', INTERVAL 10 DAY) → 2024-12-17
DATE(date)	DATE	DATE형의 날짜 부분을 반환함 예 DATE('2024-12-07 01:02:03') → 2024-12-07
DATEDIFF(date1, date2)	INTEGER	DATE형의 date1 - date2 날짜 차이를 반환함 예 DATEDIFF('2024-12-17', '2024-12-07') → 10
SYSDATE	DATE	DBMS 시스템상의 오늘 날짜를 반환함 예 SYSDATE( ) → 2024-12-07 21:47:01

표 4-5 format의 주요 지정자(specifier)

인자	설명	인자	설명
%w	요일 순서(0~6, Sunday=0)	%i	분(0~59)
%W	요일(Sunday~Saturday)	%m	월 순서(01~12, January=01)
%a	요일의 약자(Sun~Sat)	%b	월 이름 약어(Jan~Dec)
%d	한 달 중 날짜(00~31)	%M	월 이름(January~December)
%j	1년 중 날짜(001~366)	%s	초(0~59)
%h	12시간(01~12)	%Y	4자리 연도
%H	24시간(00~23)	%y	4자리 연도의 마지막 2자리

뷰 생성문

```
CREATE VIEW  Vorders
AS SELECT    orderid, O.custid, name, O.bookid, bookname, saleprice, orderdate
   FROM      Customer C, Orders O, Book B
   WHERE     C.custid=O.custid and B.bookid=O.bookid;
```

```
CREATE OR REPLACE VIEW vw_Customer (custid, name, address)
AS SELECT     custid, name, address
   FROM       Customer
   WHERE      address LIKE '%영국%';
```

```
DROP VIEW vw_Customer;
```

```
CREATE [UNIQUE] INDEX [인덱스이름]
ON 테이블이름 (컬럼 [ASC | DESC] [{, 컬럼 [ASC | DESC]} ...])[;]
```

```
CREATE INDEX ix_Book ON Book(bookname);
```

```
SHOW INDEX FROM Book;
```

```
ANALYZE TABLE Book;
```

```
DROP INDEX ix_Book ON Book;
```